

Auth Diagnostic Report

1. Summary

This report is focused only on the login/authentication failure observed in the app. The error message is:

- Could not parse request body as JSON: unexpected end of JSON input

That failure points to a broken JSON POST in the login flow, most likely in the OAuth PKCE callback path.

2. Relevant files

- `server.js`
- `routes/auth.js`
- `views/pages/login.ejs`
- `views/pages/auth-callback.ejs`

3. Affected flow

The failure appears during the OAuth / PKCE login callback rather than the initial provider redirect. The relevant flow is:

1. User clicks Google/Discord/Kick login on `views/pages/login.ejs`
2. `oauthSignIn()` generates a PKCE verifier and challenge, stores the verifier in `localStorage`, and redirects to Supabase authorize
3. Supabase redirects back to `/auth/callback` or `/auth/v1/callback`
4. `views/pages/auth-callback.ejs` reads `?code=` and posts JSON to `/auth/exchange`
5. The request fails with the JSON parse error and the user is returned to `/login`

4. Code analysis

4.1 `views/pages/auth-callback.ejs`

This file is the critical junction. It runs client-side callback logic and posts to `/auth/exchange`:

- It reads `code` from the URL query string.
- It reads `wage_pkce_verifier` from `localStorage`.
- It sends a request:

```
fetch('/auth/exchange', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ code: code, code_verifier: verifier }),
})
```

If this POST contains an empty body, malformed JSON, or if the route rejects it before returning JSON, the client-side error path may unwind poorly.

4.2 `routes/auth.js` `/auth/exchange`

This route expects a valid JSON body:

```
const { code, code_verifier } = req.body;
if (!code || !code_verifier) {
  return res.status(400).json({ error: 'code and code_verifier are required' });
}
```

It relies on `express.json()` to parse the request body. If the incoming request is malformed, Express will generate a body-parser error instead of this route-specific JSON response.

The handler then calls Supabase token endpoint directly:

```
const tokenResp = await fetch(`${process.env.SUPABASE_URL}/auth/v1/token?grant_type=pkce`, {
  method: 'POST',
  headers: {
    'Content-Type': 'application/x-www-form-urlencoded',
    'apikey': process.env.SUPABASE_ANON_KEY,
    'Authorization': `Bearer ${process.env.SUPABASE_ANON_KEY}`,
  },
  body: new URLSearchParams({ code, code_verifier }),
});
```

4.3 `server.js`

The app config includes `express.json()` and `express.urlencoded()`:

```
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
```

That is correct for JSON parsing, but there is no custom error handler to translate JSON parse failures into clean JSON responses. This means raw parse errors can be returned as plain text, exactly matching the observed string.

5. Likely root causes

5.1 Client POST to `/auth/exchange` is malformed or missing

The most direct cause of the observed error is that `/auth/exchange` receives a request whose `Content-Type` is `application/json` but the body is not valid JSON. Possible triggers:

- `JSON.stringify({ code, code_verifier })` is called with `code` or `code_verifier` undefined.
- The browser aborts or truncates the request before body data is sent.
- The request is accidentally sent as an empty body by the callback page.

5.2 Missing or invalid PKCE verifier

If `localStorage.getItem('wage_pkce_verifier')` returns `null`, the code does not post to `/auth/exchange` and instead shows a session-expired error. That is not this exact error, but the verifier path is still a likely weak point.

5.3 Upstream Supabase token exchange rejection

The route sends a direct PKCE token request to Supabase with only `code` and `code_verifier`. If Supabase requires additional parameters such as `redirect_uri` or a matching `redirect` value for PKCE, the token exchange may fail before returning JSON. That failure can manifest as a generic parse or server error.

5.4 Body parser error on the server

Express's built-in JSON parser throws on invalid JSON input and returns a plain text error string:

- `Could not parse request body as JSON: unexpected end of JSON input`

That is exactly the observed message, which strongly suggests the incoming request body is empty or malformed.

6. Confirmed unreliable areas

6.1 `auth-callback.ejs` error reporting

The callback page directly surfaces query-string errors from Supabase using:

```
var errParam = params.get('error') || params.get('error_description');
if (errParam) showErr(errParam);
```

That means any upstream Supabase error text is shown directly to the user without normalization.

6.2 Unused or duplicate session routes

There are multiple auth session routes in `routes/auth.js`:

- `/auth/token-session`
- `/auth/session`
- `/auth/exchange`

The current callback page uses `/auth/exchange`, but other code and comments suggest alternative flows. This duplication makes the auth flow harder to reason about and easier to misconfigure.

7. Recommended fixes

7.1 Add robust request validation and logging

- In `auth-callback.ejs`, log or guard the JSON body before posting to `/auth/exchange`.
- Add a fallback before `fetch` to confirm both `code` and `code_verifier` are non-empty.

7.2 Add a JSON parse error handler in `server.js`

Add an Express error middleware after body parsing:

```
app.use((err, req, res, next) => {
  if (err?.type === 'entity.parse.failed') {
    return res.status(400).json({ error: 'Invalid JSON body' });
  }
  next(err);
});
```

This converts raw body-parser failures into structured JSON and prevents opaque plain-text messages from propagating through the client.

7.3 Confirm Supabase PKCE requirements

- Verify whether the Supabase token endpoint requires `redirect_uri` or `client_id` in the PKCE token request.
- If required, include `redirect_uri` exactly as the original authorize request value.

7.4 Clean up callback path duplication

Focus the auth callback on a single flow:

- magic-link → `/auth/verify`
- OAuth PKCE → `/auth/exchange`

Remove or deprecate unused legacy `/auth/session` handling if it is not actually in use.

8. Suggested immediate debugging steps

1. Reproduce the failing login flow while watching server logs.
2. Inspect `/auth/exchange` request bodies and confirm `code` + `code_verifier` values.
3. Check whether `routes/auth.js` logs `SUPABASE STATUS` and raw response for the failed exchange.
4. If the server never logs the exchange attempt, the problem is most likely a malformed request body before the route is reached.
5. If the server logs the exchange attempt and Supabase returns a token error, the bug is in the PKCE token exchange request format.

9. Conclusion

The observed error is almost certainly caused by an invalid or empty JSON request body on the PKCE callback exchange path. The core failure point is the client-side POST to `/auth/exchange` combined with an Express JSON parser that currently returns raw text on parse failure.

Fixing this should begin with hardening the callback page and adding a structured body-parser error handler.